

PDF Merger API

Bulut Mimarilerinde Test Mühendisliği Konteyner Mimarisi
Final Projesi

Demirhan Çakır - 150820513

Proje & İşleyiş

Proje

- Birden fazla PDF'i tek dosyada birleştiren; üretime hazır, cloud-native bir REST servisi.

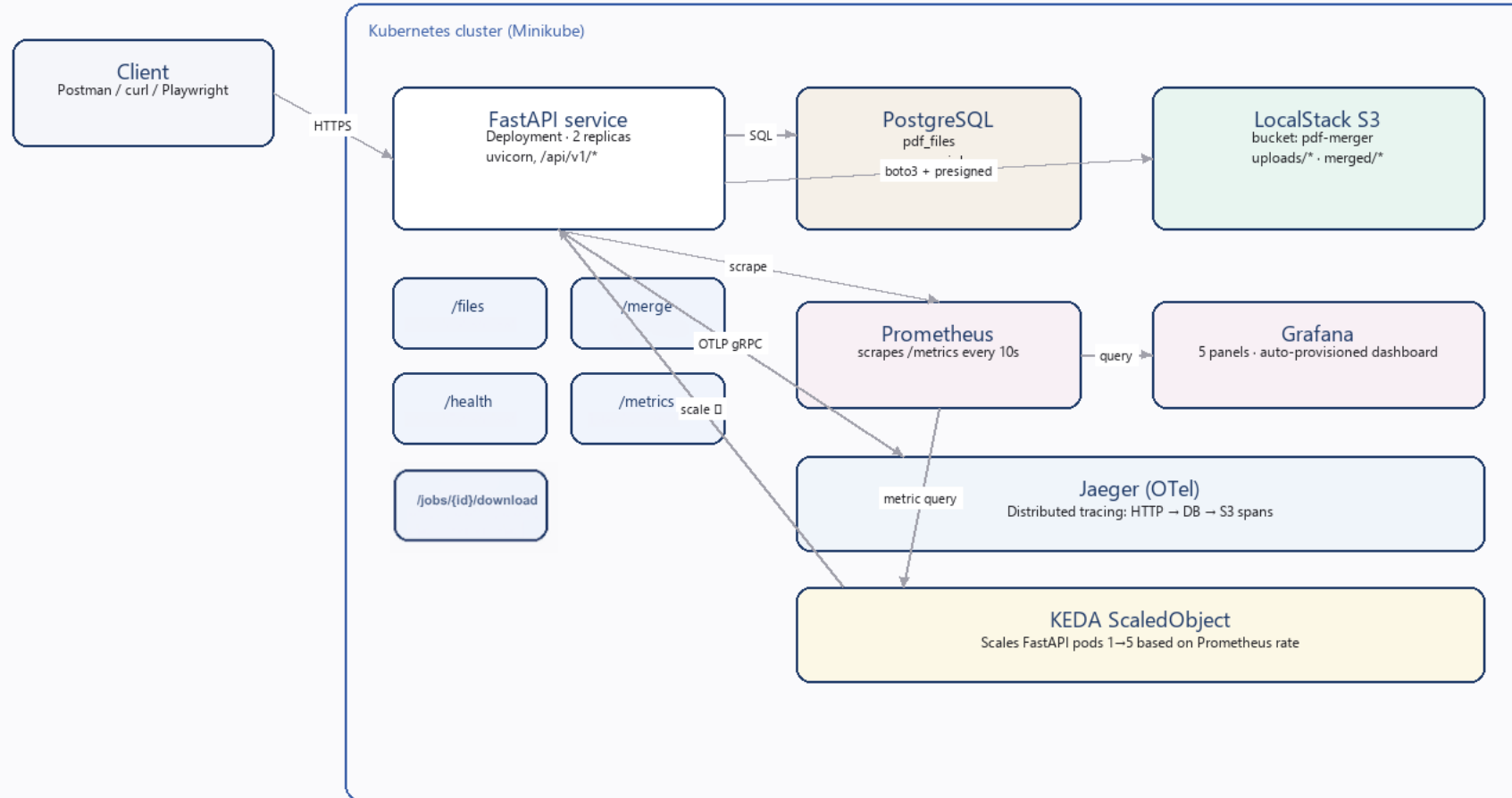
İşleyiş

- job tabanlı API + S3 nesne deposu + Kubernetes üzerinde otomatik ölçeklenen ve gözlemlenebilir bir mimari.

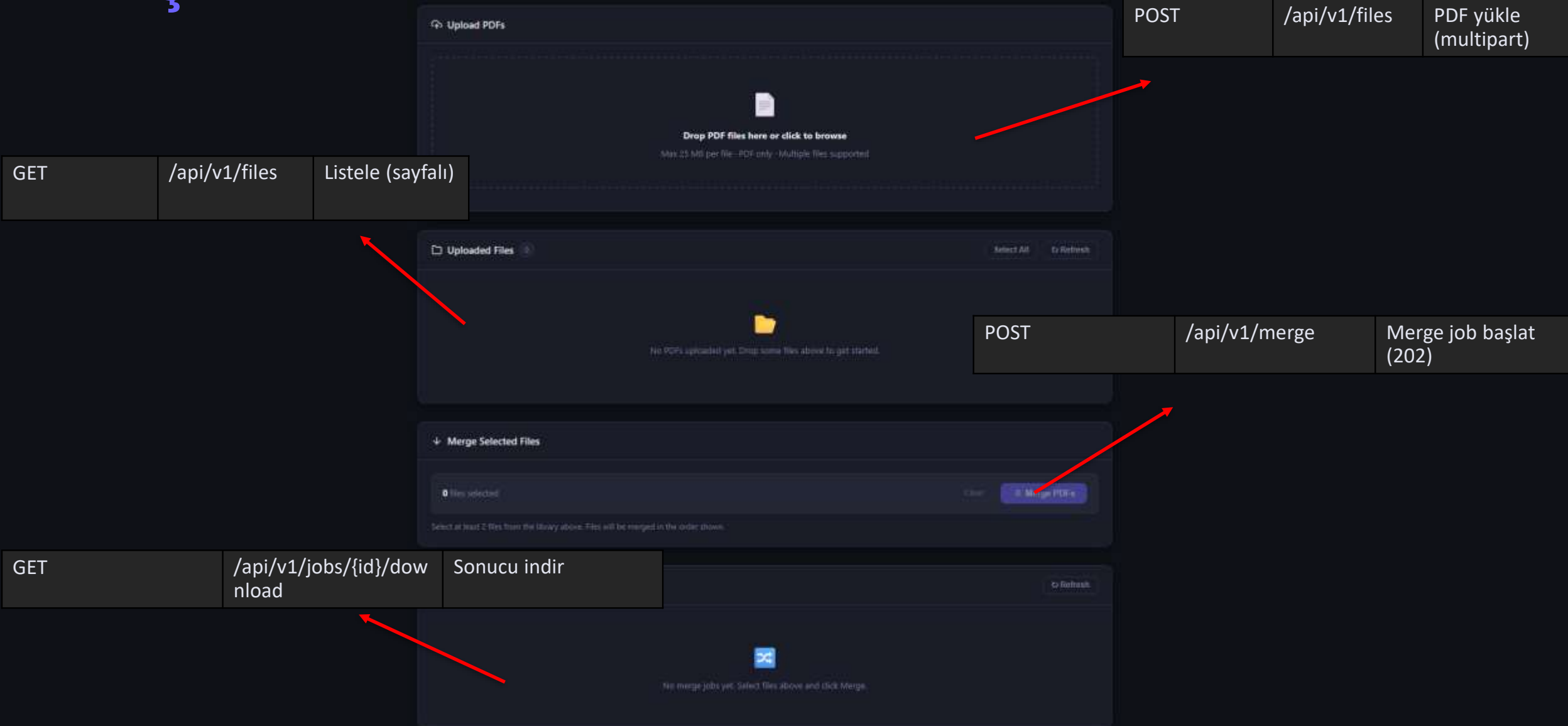
Mimari

PDF Merger API — Architecture

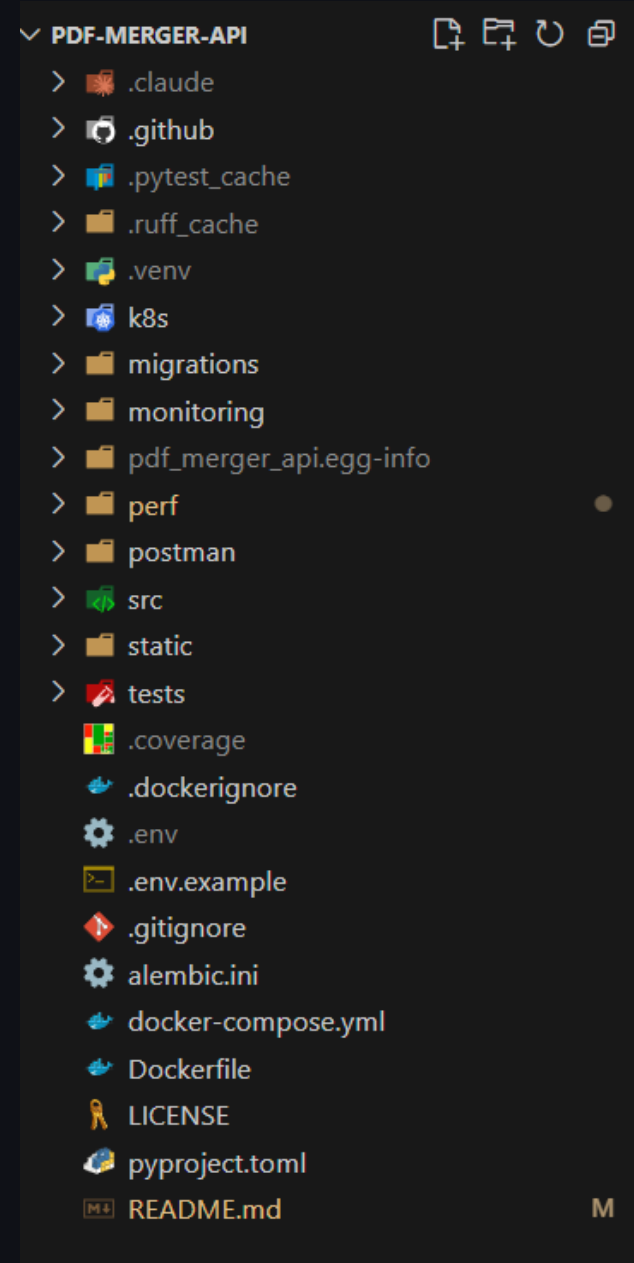
FastAPI · PostgreSQL · LocalStack S3 · Kubernetes · Prometheus/Grafana · OpenTelemetry



Akış



Proje Dosya Dizini



Kullanılan Teknolojiler

- API: FastAPI · Pydantic v2 · SQLAlchemy 2.0
- Veri: PostgreSQL 16 · LocalStack S3 (boto3) · pypdf
- Paketleme: Multi-stage Dockerfile · docker-compose
- Orkestrasyon: Kubernetes (Deployment/Service/ConfigMap/Secret) · KEDA event-driven autoscale
- Test: pytest (+Testcontainers) · Factory Boy/Faker · Postman/Newman · Playwright (E2E) · k6 (perf)
- Gözlemlenebilirlik: Prometheus · Grafana · OpenTelemetry → Jaeger
- CI/CD: GitHub Actions — lint → test → build → deploy → smoke

API & Veri Modeli

Entity'ler: PdfFile (id, filename, s3_key, size, page_count) · MergeJob (id, status, source_file_ids, output_s3_key, timestamps)

Durum: pending → processing → completed | failed

Method	Path	Açıklama
POST	/api/v1/files	PDF yükle (multipart)
GET	/api/v1/files	Listele (sayfalı)
POST	/api/v1/merge	Merge job başlat (202)
GET	/api/v1/jobs/{id}	Durum sorgula
GET	/api/v1/jobs/{id}/download	Sonucu indir
GET	/health · /metrics	Sağlık & metrik

Test & Kalite Stratejisi

- Unit: config · models · schemas · merge_pdf_bytes (saf fonksiyon)
- Integration: Testcontainers ile gerçek Postgres + LocalStack S3'e karşı tam upload→merge→download akışı
- Test verisi: Factory Boy + Faker (PdfFileFactory, MergeJobFactory)
- API sözleşmesi: Postman/Newman — 7 istek, CI'da koşar
- E2E: Playwright — Swagger UI + canlı endpoint'ler (5 senaryo)
- Kapsam: CI'da --cov-fail-under=70 → eşiğin altı = build fail
- Lint: ruff (check + format)

Gözlemlenebilirlik & Ölçeklenme

- Metrikler: /metrics (Prometheus), ServiceMonitor ile scrape
- Grafana panelleri (4): Request rate · Endpoint bazlı rate · Replica sayısı · p95 latency (histogram_quantile)
- Tracing: OpenTelemetry → Jaeger
- Autoscale (KEDA): $\text{sum}(\text{rate}(\text{http_requests_total}[2\text{m}])) \geq 10 \text{ req/s} \rightarrow 1 \rightarrow 5 \text{ replika}$
- Performans (k6): ramp 1 → 50 VU, ~100 s

CI/CD Pipeline

```
lint (ruff) -> test (pytest+cov>=70) -> build (docker) -> deploy (minikube) -> smoke (Newman + k6)
```

- Tek workflow; her aşama bir öncekine bağımlı — biri düşerse pipeline kırmızı.
- Image artifact olarak taşınır → minikube'e yüklenir → manifest'ler uygulanır → smoke ile canlılık doğrulanır.

Teşekkürler Ederim :)